REPORTE OFICIAL DE AUDITORÍA Y DESARROLLO

DISGLOBAL TRACKING

Manual Completo de Arquitectura de Software, Diccionario de Datos y Guía Metodológica de Exposición para la Auditoría de Control de Seriales de Puntos de Venta (POS).



PREPARADO PARA

Auditoría de Sistemas & Mesa Técnica

PLATAFORMA

React 19 + TypeScript + IndexedDB

FECHA DE EMISIÓN

26 de Mayo, 2026

ESTADO DE CLASIFICACIÓN

Confidencial / Uso Interno

Índice de Contenidos

1. Resumen Ejecutivo e Introducción Operativa ..... Pág. 3

2. Arquitectura del Software y Pila Tecnológica ..... Pág. 4

3. Capa de Datos y Persistencia Local (IndexedDB) ..... Pág. 5

4. El Motor de Conciliación: Cruce de Seriales ..... Pág. 6

5. Diccionario de Datos Completo (types.ts) ..... Pág. 8

6. Guía Práctica de Exposición para Auditorías ..... Pág. 9

Nota para el Auditor: Este documento está redactado con un enfoque técnico-logístico estructurado para verificar la integridad del almacenamiento, la resiliencia offline ante fallos de conectividad, y la complejidad algorítmica de los cruces masivos de inventario.

1. Resumen Ejecutivo e Introducción Operativa

El sistema **Disglobal Tracking** es una aplicación web de nivel corporativo diseñada con el propósito de resolver una de las ineficiencias más comunes en la cadena de suministro y soporte técnico de dispositivos de cobro móvil: la discrepancia de seriales entre la información transmitida digitalmente (vía correo o bases de datos como IMS/CMS) y los equipos físicos recibidos en los talleres o almacenes.

El principal problema que este software mitiga es el "extravío administrativo", donde un serial se registra como despachado pero el operador de almacén recibe físicamente un serial diferente o deteriorado. A través de un motor de cruce en tiempo real compatible con lectores ópticos de códigos de barra (pistolas de escaneo), el sistema permite a los auditores consolidar el inventario de manera instantánea, registrando las discrepancias y actualizando la base de datos local de forma inmediata.

Objetivos Clave de Auditoría Logística

- **Integridad de Entrada:** Garantizar que cada serial ingresado cumpla con la estructura requerida y posea un historial único dentro del sistema.
- **Detección de Discrepancias:** Visualizar en tiempo real qué dispositivos están "Solo en Correo" (en tránsito administrativo pero ausentes físicamente) o "Solo en Físico" (equipos recibidos que no figuran en las órdenes de despacho).
- **Trazabilidad y Resiliencia:** Ofrecer un registro histórico persistente a nivel de dispositivo que sea inmune a pérdidas de conexión a red gracias a su arquitectura offline-first.



FIGURA 1.1: FLUJO LOGÍSTICO DE PROCESAMIENTO Y AUDITORÍA DE SERIALES.

2. Arquitectura del Software y Pila Tecnológica

La aplicación está construida sobre una arquitectura moderna basada en componentes React. Se prescinde de dependencias de servidor complejas para el funcionamiento operativo ordinario, consolidando una aplicación estática de alto rendimiento capaz de ejecutarse en navegadores web de manera aislada (Edge, Chrome, Firefox).

Componentes Tecnológicos del Ecosistema

TECNOLOGÍA	VERSIÓN	PROPÓSITO EN LA APLICACIÓN
React	19.0.1	Manejo del ciclo de vida de la UI, estados reactivos locales y rendering eficiente mediante Reconciliation de componentes.
TypeScript	~5.8.2	Tipado estático estricto. Define contratos para objetos como seriales, perfiles de usuario y cruces, previniendo errores en tiempo de ejecución.
Vite	6.2.3	Herramienta de compilación ultra rápida. Agiliza el desarrollo y genera bundles minificados de alta eficiencia de carga.
Tailwind CSS	4.1.14	Estilización modular y responsiva del panel. Aporta la estética premium con esquemas oscuros (Slate-950/Slate-900) y acentos en azul neón.
IndexedDB	Estándar W3C	Base de datos transaccional no relacional en el cliente para persistencia de más de 100,000 registros de manera local y veloz.

Estructura de Módulos y Flujo de Control

El código fuente está estructurado de manera modular para garantizar la escalabilidad y mantenibilidad por parte del personal de TI:

- `src/main.tsx`: Punto de entrada del bundle React. Monta el componente raíz en el DOM HTML5.
- `src/App.tsx`: Coordinador central. Gestiona el estado maestro de los seriales, los filtros unificados de fecha/búsqueda, los menús de navegación, los eventos de importación/exportación de archivos y la hidratación inicial de datos.

- `src/types.ts`: Archivo central de tipado donde se declaran los enums y las interfaces del dominio logístico.
- `src/db.ts`: Módulo encapsulado encargado de la persistencia física a nivel del navegador utilizando la API nativa de IndexedDB.
- `src/data.ts`: Archivo de datos semilla. Contiene el catálogo predefinido de marcas de POS (Sunmi, Verifone, PAX, Zebra, Ingenico) y los algoritmos de resolución y detección automática de marcas según el prefijo del serial.



FIGURA 2.1: DIAGRAMA DE CAPAS DEL SISTEMA DISGLOBAL TRACKING.

3. Capa de Datos y Persistencia Local (IndexedDB)

Una de las directrices más importantes de la auditoría técnica es verificar cómo y dónde se almacenan los datos ingresados por el usuario. En Disglobal Tracking, los datos de inventario no se envían a servidores de terceros de manera predeterminada, mitigando brechas de seguridad de información confidencial.

El archivo `src/db.ts` implementa una base de datos local usando **IndexedDB**. Esto otorga al sistema la capacidad de almacenar de manera confiable millones de caracteres, superando el límite convencional de 5MB que impone la especificación clásica de `localStorage`.

Estrategia de Migración y Resiliencia

Para asegurar una transición suave si el cliente poseía datos guardados en versiones anteriores de la plataforma, el sistema incluye un mecanismo automático de migración de datos. Al iniciar la aplicación:

1. Se abre una conexión a IndexedDB con el nombre `disglobal_tracking_db`.
2. Si no existe el almacenador de datos (Object Store) denominado `keyvalue_store`, se crea en el evento `onupgradeneeded`.
3. Al leer una clave mediante `getDBItem`, si IndexedDB no posee registros, el script verifica si existen remanentes en el antiguo `localStorage`.
4. Si los encuentra, los deserializa, los guarda en IndexedDB, y los elimina de `localStorage` para liberar cuota de espacio del navegador.

Implementación del Motor de Almacenamiento

```
// Fragmento clave de src/db.ts para la migración transparente
export async function getDBItem<T>(key: string, defaultValue: T): Promise<T> {
  try {
    const db = await getDB();
    return new Promise((resolve) => {
      const transaction = db.transaction(STORE_NAME, 'readonly');
      const store = transaction.objectStore(STORE_NAME);
      const request = store.get(key);

      request.onsuccess = async () => {
        const result = request.result;
        if (result !== undefined) {
          resolve(result as T);
        } else {
          // Intento de migración desde localStorage heredado
          const localRaw = localStorage.getItem(key);
          if (localRaw !== null) {
            try {
              const parsed = JSON.parse(localRaw) as T;
              await setDBItem(key, parsed); // Migra a IndexedDB
              localStorage.removeItem(key); // Limpia localStorage antiguo
              resolve(parsed);
            } catch (e) {
              resolve(defaultValue);
            }
          } else {
            resolve(defaultValue);
          }
        }
      };
      // ... manejo de errores y fallback
    });
  } catch (error) {
    // Si IndexedDB falla (ej. navegación privada estricta), cae a localStorage
    const localRaw = localStorage.getItem(key);
    // ...
  }
}
```

> **Garantía de Auditoría:** El sistema cuenta con resiliencia de doble capa. Si IndexedDB llega a fallar debido a restricciones severas de seguridad del navegador, la capa de datos automáticamente conmuta hacia `localStorage` de forma transparente para el usuario final.

4. El Motor de Conciliación: Cruce de Seriales

El núcleo algorítmico y operativo del sistema reside en el componente `src/components/CruceSeriales.tsx`. Este componente permite realizar la conciliación entre la lista digital de despacho (correo) y los seriales leídos físicamente en el taller.

Complejidad Algorítmica y Rendimiento O(1)

En inventarios a gran escala, realizar búsquedas dentro de arreglos mediante funciones nativas como `Array.find()` o `Array.includes()` resulta en una complejidad algorítmica de **O(N)**. Si se realiza esta búsqueda en un bucle sobre otra lista de tamaño M, la complejidad combinada asciende a **O(N × M)**. Para listas de 100,000 seriales, esto representaría hasta 10 mil millones de operaciones, lo cual bloquea el navegador y causa un desbordamiento de pila o cuelgue de la UI.

Para evitar esto, **Disglobal Tracking** implementa un indexado de búsqueda en tiempo constante **O(1)** utilizando mapas asociativos generados reactivamente a través de `useMemo` de React. Esto reduce el costo computacional total a **O(N + M)** operaciones, permitiendo procesar conjuntos masivos en milisegundos.

```
// Creación de índice Hash O(1) en src/components/CruceSeriales.tsx
const maestroMap = useMemo(() => {
  const map: Record<string, CruceItem> = {};
  for (let i = 0; i < datosMaestro.length; i++) {
    const item = datosMaestro[i];
    if (item && item.serial) {
      map[item.serial.trim().toUpperCase()] = item;
    }
  }
  return map;
}, [datosMaestro]);

// La búsqueda del estado del serial se realiza de forma directa en el mapa hash:
const lookupValidador = (serial: string) => {
  const cleanSerial = serial.trim().toUpperCase();

  // Búsqueda instantánea O(1)
  const hit = maestroMap[cleanSerial];
  if (hit) return hit;

  // Si no está registrado en el maestro, retorna estado no aplicable
  return { serial: cleanSerial, status: 'N#A' };
};
```

Lógica de Conjuntos Matemáticos en el Cruce

El componente realiza una intersección y diferencia de conjuntos (Álgebra de Boole) para clasificar dinámicamente los seriales ingresados:

- **Coincidencias (Intersección):** Elementos presentes tanto en la lista de correo como en la física. `A ∩ B`.
- **Solo Correo (Diferencia A - B):** Elementos que figuran en el correo electrónico pero que no han sido escaneados físicamente. Representa equipos faltantes.
- **Solo Físico (Diferencia B - A):** Equipos que fueron escaneados físicamente pero no estaban registrados en el correo digital de despacho. Representa excedentes o errores de asignación.

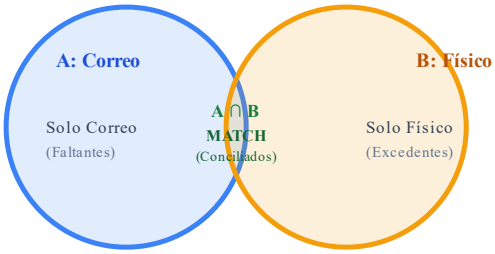


FIGURA 4.1: DIAGRAMA DE VENN DE LA LÓGICA DE CONCILIACIÓN.

Integración Nativa con Pistolas de Códigos de Barra

El sistema cuenta con un campo de escaneo especializado que aprovecha la naturaleza del emulador de teclado de los lectores de códigos de barra. Al escanear un serial, el hardware envía la cadena de caracteres seguida de la tecla `Enter`. El sistema intercepta el evento `KeyDown`, evita el envío del formulario, agrega el serial inmediatamente al listado físico si no estaba ya presente, y limpia el campo enfocando el cursor nuevamente de forma automática en 30 milisegundos para permitir escaneos continuos ultra rápidos.

5. Diccionario de Datos Completo (types.ts)

La solidez estructural de los datos en el sistema está respaldada por la tipificación de TypeScript en el archivo `src/types.ts`. A continuación, se presenta la especificación detallada de los objetos de datos utilizados en la aplicación, de vital importancia para auditorías de sistemas relacionales.

Estructura de la Entidad Principal: POSItem

Representa el estado logístico, comercial y físico de cada terminal que entra al almacén de Disglobal.

CAMPO	TIPO DE DATO	DESCRIPCIÓN Y RESTRICCIÓN LOGÍSTICA
`id`	`number`	Identificador único autoincrementable generado al registrar.
`serial`	`string`	Número de serie único del fabricante de la terminal (Clave primaria lógica).
`date`	`string` (ISO)	Fecha de recepción original del equipo (Formato: AAAA-MM-DD).
`displayDate`	`string`	Fecha formateada para la UI (Formato: DD/MM/AAAA).
`status`	`POSStatus`	Estado operativo en el inventario. Puede tomar uno de los valores definidos abajo.
`remitente`	`string`	Identificador del remitente, cliente o entidad logística emisora.
`rif`	`string` (Opcional)	Registro de Información Fiscal del cliente asignado al terminal.
`razonSocial`	`string` (Opcional)	Nombre legal o comercial de la empresa cliente.
`destino`	`string` (Opcional)	Destino físico asignado para despacho (ej. Maracay, Caracas).
`simNumber`	`string` (Opcional)	Número identificador del chip celular (SIM Card) asociado si aplica.
`operadora`	`'MOVISTAR' | 'MOVILNET' | 'DIGITEL' | ''`	Empresa de telecomunicaciones asignada al canal celular del dispositivo.
`modeloNegocio`	`'BASICO' | 'COMODATO' | ''`	Modalidad contractual comercial de la entrega del dispositivo.

El Dominio POSStatus

El tipo `POSStatus` restringe los estados que un terminal puede adquirir en el sistema para garantizar la coherencia semántica:

- **PENDIENTE**: Equipo ingresado que está a la espera de asignación técnica o revisión.

- `ACTIVO`: Dispositivo en pleno funcionamiento y asignado formalmente.
- `SUSPENDIDO`: Retenido administrativamente por falta de pago o investigación.
- `CANCELADO`: Devuelto o descartado administrativamente.
- `N#A`: Estado especial de descarte ("No Aplica") asignado automáticamente al no coincidir con el IMS maestro.
- `SIN ESTATUS`: Estado neutral transitorio asignado al importar de fuentes no homologadas.
- `RECUPERADO`: Equipos rescatados de campo por retiro comercial.

6. Guía Práctica de Exposición para Auditorías

Al presentar este software ante un comité técnico, junta directiva o auditor externo de sistemas, es fundamental estructurar la demostración en base a las capacidades clave del sistema. Esta guía provee un guión estructurado de exposición y responde preventivamente a preguntas críticas de auditoría.

Guión de Demostración Operativa en Vivo

1. **Demostración de Carga Masiva (Bases Maestras):**

   Ingrese a la pestaña *Cruce de Seriales* y abra el modal **IMS** o **CMS**. Pegue un listado de ejemplo de 5,000 registros simulando una planilla Excel. Presione confirmar para demostrar cómo la UI indexa instantáneamente sin colgar el navegador, gracias al mapa hash O(1).

2. **Simulación de Escaneo Físico y Conciliación:**

   Pegue una lista de seriales en la columna *Seriales desde Correo*. Luego pase a la columna *Seriales Físicos* y simule lecturas activando el modo visual y escribiendo seriales (o usando una pistola de códigos de barra). Muestre la clasificación inmediata en badges de color verde (`✓ MATCH`) o gris (`SIN MATCH`).

3. **Prueba de Resiliencia (Simulación de Caída de Red):**

   Abra las herramientas de desarrollo del navegador (F12), configure la red en modo *Offline* (Desconectado) y recargue la página. Muestre cómo la aplicación sigue cargando con total normalidad, permitiendo buscar, auditar y guardar cambios gracias a la persistencia local en IndexedDB.

Preguntas Frecuentes y Respuestas Técnicas de Auditoría

Q1: ¿Qué medidas de seguridad protegen la base de datos local contra accesos de otras webs?

Respuesta: IndexedDB está regido estrictamente por la política de seguridad del mismo origen (*Same-Origin Policy*) implementada por los navegadores modernos. Ninguna otra página web o dominio puede acceder a la base de datos `disglobal_tracking_db` creada por esta aplicación, garantizando aislamiento total.

Q2: ¿Cómo maneja el sistema la colisión de identificadores (IDs) al importar archivos CSV?

Respuesta: El sistema posee una función desinfectadora (`sanitizeSerials`) y lógica interna en la función de importación. Si el archivo CSV importado contiene IDs duplicados o vacíos, la aplicación recalcula dinámicamente un correlativo secuencial basándose en el ID máximo actual de la base de datos activa más uno. Esto garantiza la integridad y previene la sobrescritura accidental de registros históricos.

Q3: ¿Cuál es el límite máximo de almacenamiento soportado localmente?

Respuesta: El límite depende del almacenamiento disponible en el disco duro del usuario y la cuota asignada por el navegador para el origen. En sistemas modernos con Windows, Chrome o Edge permiten que un origen utilice hasta el 60% del espacio libre del disco para IndexedDB. Esto equivale a decenas de millones de registros de seriales sin degradación de rendimiento.